```
# Electricity Peak Analysis - Dorm Hourly Meter Data
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from datetime import datetime, timedelta
import warnings
warnings.filterwarnings('ignore')
```

1. Data Collection - Simulating Hourly Meter Data from Dorms

We'll simulate realistic hourly electricity consumption data from multiple dorms over the past 2 weeks.

```
def generate_hourly_dorm_data(num_days=14, num_dorms=5):
    """
    Generate realistic hourly electricity consumption data for multiple dorms

    Parameters:
    - num_days: Number of days of historical data
    - num_dorms: Number of dormitories to simulate

    Returns:
    - DataFrame with hourly consumption data
    """
    # Generate datetime range
    end_date = datetime.now()
    start_date = end_date - timedelta(days=num_days)
    date_range = pd.date_range(start=start_date, end=end_date, freq='h')

    # Create empty dataframe
    data = []

    for dorm_id in range(1, num_dorms + 1):
        for dt in date_range:
            hour = dt.hour
            day_of_week = dt.dayofweek  # Monday=0, Sunday=6

            # Base consumption pattern (higher during evening)
            base_consumption = 50 + 30 * np.sin((hour - 6) * np.pi / 12)

            # Evening peak (6 PM - 10 PM)
            if 18 <= hour <= 22:
                evening_peak = 40 + 20 * np.random.random()
            else:
                evening_peak = 0

            # Weekend pattern (slightly lower)
            weekend_factor = 0.85 if day_of_week >= 5 else 1.0

            # Morning peak (7 AM - 9 AM)
            if 7 <= hour <= 9:
                morning_peak = 25 + 10 * np.random.random()
            else:
                morning_peak = 0

            # Night low consumption (12 AM - 5 AM)
            if 0 <= hour <= 5:
                night_factor = 0.4
            else:
                night_factor = 1.0

            # Calculate total consumption with random noise
            consumption = (base_consumption + evening_peak + morning_peak) * weekend_factor * night_factor
            consumption += np.random.normal(0, 5)  # Add noise
            consumption = max(consumption, 10)  # Minimum consumption

            data.append({
                'timestamp': dt,
                'dorm_id': f'Dorm_{dorm_id}',
                'hour': hour,
                'day_of_week': day_of_week,
                'consumption_kwh': round(consumption, 2)
            })
```

```
    df = pd.DataFrame(data)
    return df

# Generate the data
print("Collecting hourly meter data from dorms...")
df_raw = generate_hourly_dorm_data(num_days=14, num_dorms=5)
print(f"\nData collected: {len(df_raw)} records from {df_raw['dorm_id'].nunique()} dorms")
print(f"Date range: {df_raw['timestamp'].min()} to {df_raw['timestamp'].max()}")
print(f"\nFirst few records:")
print(df_raw.head(10))
```

```
Collecting hourly meter data from dorms...

Data collected: 1685 records from 5 dorms
Date range: 2026-02-05 11:06:10.850909 to 2026-02-19 11:06:10.850909

First few records:
                  timestamp dorm_id  hour  day_of_week  consumption_kwh
0 2026-02-05 11:06:10.850909  Dorm_1    11            3            83.13
1 2026-02-05 12:06:10.850909  Dorm_1    12            3            82.49
2 2026-02-05 13:06:10.850909  Dorm_1    13            3            79.04
3 2026-02-05 14:06:10.850909  Dorm_1    14            3            82.63
4 2026-02-05 15:06:10.850909  Dorm_1    15            3            68.20
5 2026-02-05 16:06:10.850909  Dorm_1    16            3            60.56
6 2026-02-05 17:06:10.850909  Dorm_1    17            3            51.34
7 2026-02-05 18:06:10.850909  Dorm_1    18            3           105.04
8 2026-02-05 19:06:10.850909  Dorm_1    19            3            78.00
9 2026-02-05 20:06:10.850909  Dorm_1    20            3            88.56
```

2. Data Aggregation and Preprocessing

Aggregate consumption across all dorms by hour to get total campus consumption.

```
# Aggregate consumption across all dorms by timestamp
df_aggregated = df_raw.groupby('timestamp').agg({
    'consumption_kwh': 'sum',
    'hour': 'first',
    'day_of_week': 'first'
}).reset_index()

df_aggregated.columns = ['timestamp', 'total_consumption_kwh', 'hour', 'day_of_week']

print("Aggregated data summary:")
print(df_aggregated.describe())
print(f"\nTotal records: {len(df_aggregated)}")
```

```
Aggregated data summary:
                        timestamp  total_consumption_kwh        hour  \
count                         337             337.000000  337.000000
mean   2026-02-12 11:06:10.850908             289.442285   11.498516
min    2026-02-05 11:06:10.850909              50.000000    0.000000
25%    2026-02-08 23:06:10.850909              92.580000    6.000000
50%    2026-02-12 11:06:10.850909             344.030000   11.000000
75%    2026-02-15 23:06:10.850909             403.770000   17.000000
max    2026-02-19 11:06:10.850909             527.530000   23.000000
std                           NaN             153.181318    6.922240

       day_of_week
count        337.0
mean           3.0
min            0.0
25%            1.0
50%            3.0
75%            5.0
max            6.0
std            2.0

Total records: 337
```

3. Moving Average Smoothing

Apply moving average smoothing to reduce noise and identify underlying trends.

```
def apply_moving_average(df, column, window_sizes=[3, 6, 12]):
    """
    Apply moving average smoothing with different window sizes

    Parameters:
    - df: DataFrame with time series data
    - column: Column name to smooth
    - window_sizes: List of window sizes in hours
```

```
    Returns:
    - DataFrame with additional smoothed columns
    """
    df_smooth = df.copy()

    for window in window_sizes:
        col_name = f'{column}_ma{window}h'
        df_smooth[col_name] = df_smooth[column].rolling(window=window, center=True).mean()

    return df_smooth

# Apply moving average smoothing
print("Applying moving average smoothing...")
df_smoothed = apply_moving_average(
    df_aggregated,
    'total_consumption_kwh',
    window_sizes=[3, 6, 12, 24]
)

print("\nSmoothed data preview:")
print(df_smoothed[['timestamp', 'total_consumption_kwh',
                   'total_consumption_kwh_ma3h', 'total_consumption_kwh_ma6h',
                   'total_consumption_kwh_ma12h', 'total_consumption_kwh_ma24h']].head(30))
```

```
Applying moving average smoothing...

Smoothed data preview:
                    timestamp  total_consumption_kwh  \
0  2026-02-05 11:06:10.850909                 412.44
1  2026-02-05 12:06:10.850909                 413.90
2  2026-02-05 13:06:10.850909                 405.06
3  2026-02-05 14:06:10.850909                 386.16
4  2026-02-05 15:06:10.850909                 358.36
5  2026-02-05 16:06:10.850909                 324.20
6  2026-02-05 17:06:10.850909                 270.68
7  2026-02-05 18:06:10.850909                 527.53
8  2026-02-05 19:06:10.850909                 456.10
9  2026-02-05 20:06:10.850909                 427.22
10 2026-02-05 21:06:10.850909                 385.53
11 2026-02-05 22:06:10.850909                 362.32
12 2026-02-05 23:06:10.850909                 108.52
13 2026-02-06 00:06:10.850909                  61.16
14 2026-02-06 01:06:10.850909                  63.81
15 2026-02-06 02:06:10.850909                  66.80
16 2026-02-06 03:06:10.850909                  82.53
17 2026-02-06 04:06:10.850909                  60.92
18 2026-02-06 05:06:10.850909                  80.48
19 2026-02-06 06:06:10.850909                 235.23
20 2026-02-06 07:06:10.850909                 420.38
21 2026-02-06 08:06:10.850909                 482.37
22 2026-02-06 09:06:10.850909                 463.94
23 2026-02-06 10:06:10.850909                 387.98
24 2026-02-06 11:06:10.850909                 393.52
25 2026-02-06 12:06:10.850909                 386.03
26 2026-02-06 13:06:10.850909                 405.19
27 2026-02-06 14:06:10.850909                 374.77
28 2026-02-06 15:06:10.850909                 356.22
29 2026-02-06 16:06:10.850909                 329.17

    total_consumption_kwh_ma3h  total_consumption_kwh_ma6h  \
0                          NaN                         NaN
1                   410.466667                         NaN
2                   401.706667                         NaN
3                   383.193333                  383.353333
4                   356.240000                  359.726667
5                   317.746667                  378.665000
6                   374.136667                  387.171667
7                   418.103333                  394.015000
8                   470.283333                  398.543333
9                   422.950000                  404.896667
10                  391.690000                  377.870000
11                  285.456667                  300.141667
12                  177.333333                  234.760000
13                   77.830000                  174.690000
14                   63.923333                  124.190000
15                   71.046667                   73.956667
16                   70.083333                   69.283333
17                   74.643333                   98.295000
18                  125.543333                  157.723333
19                  245.363333                  226.985000
20                  379.326667                  290.553333
21                  455.563333                  345.063333
```

4. Extract Evening Peak Data

Extract evening peak hours (6 PM - 10 PM) for prediction modeling.

```
# Extract evening peak data (6 PM to 10 PM)
df_evening = df_smoothed[df_smoothed['hour'].between(18, 22)].copy()

# Get daily evening peak (maximum consumption during evening hours)
df_evening['date'] = df_evening['timestamp'].dt.date
daily_evening_peaks = df_evening.groupby('date').agg({
    'total_consumption_kwh': 'max',
    'total_consumption_kwh_ma6h': 'max'
}).reset_index()

daily_evening_peaks.columns = ['date', 'evening_peak_kwh', 'evening_peak_smoothed_kwh']
daily_evening_peaks['date'] = pd.to_datetime(daily_evening_peaks['date'])

print(f"Evening peak data extracted for {len(daily_evening_peaks)} days")
print("\nDaily evening peaks:")
print(daily_evening_peaks)
```

```
Evening peak data extracted for 14 days

Daily evening peaks:
         date  evening_peak_kwh  evening_peak_smoothed_kwh
0  2026-02-05            527.53                 404.896667
1  2026-02-06            485.58                 412.826667
2  2026-02-07            414.91                 345.770000
3  2026-02-08            409.16                 343.671667
4  2026-02-09            493.05                 407.273333
5  2026-02-10            489.40                 402.985000
6  2026-02-11            494.62                 400.475000
7  2026-02-12            491.81                 405.681667
8  2026-02-13            520.61                 411.563333
9  2026-02-14            424.42                 347.506667
10 2026-02-15            447.28                 349.011667
11 2026-02-16            485.40                 401.280000
12 2026-02-17            496.36                 417.771667
13 2026-02-18            511.03                 418.275000
```

5. Linear Regression Model - Predict Evening Peaks

Build a linear regression model using the past week's data to predict future evening peaks.

```
def train_peak_prediction_model(df_peaks, train_days=7):
    """
    Train linear regression model to predict evening peaks

    Parameters:
    - df_peaks: DataFrame with daily evening peaks
    - train_days: Number of days to use for training (past week)

    Returns:
    - Trained model, predictions, and metrics
    """
    # Prepare features
    df_model = df_peaks.copy()
    df_model['day_num'] = range(len(df_model))
    df_model['day_of_week'] = df_model['date'].dt.dayofweek
    df_model['is_weekend'] = (df_model['day_of_week'] >= 5).astype(int)

    # Split into training (past week) and testing (recent days)
    train_data = df_model.iloc[:-train_days]
    test_data = df_model.iloc[-train_days:]

    # Features: day number, day of week, is weekend, previous day's peak
    X_train = train_data[['day_num', 'day_of_week', 'is_weekend']].values
    y_train = train_data['evening_peak_smoothed_kwh'].values

    X_test = test_data[['day_num', 'day_of_week', 'is_weekend']].values
    y_test = test_data['evening_peak_smoothed_kwh'].values

    # Train linear regression model
    model = LinearRegression()
    model.fit(X_train, y_train)

    # Make predictions
    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)

    # Calculate metrics
    train_r2 = r2_score(y_train, y_pred_train)
    test_r2 = r2_score(y_test, y_pred_test)
```

```
    train_rmse = np.sqrt(mean_squared_error(y_train, y_pred_train))
    test_rmse = np.sqrt(mean_squared_error(y_test, y_pred_test))

    # Future predictions (next 7 days)
    last_day = df_model['day_num'].max()
    future_days = []
    for i in range(1, 8):
        future_day_num = last_day + i
        future_date = df_model['date'].max() + timedelta(days=i)
        future_dow = future_date.dayofweek
        future_weekend = 1 if future_dow >= 5 else 0
        future_days.append([future_day_num, future_dow, future_weekend])

    X_future = np.array(future_days)
    y_future = model.predict(X_future)

    future_dates = [df_model['date'].max() + timedelta(days=i) for i in range(1, 8)]

    return {
        'model': model,
        'train_data': train_data,
        'test_data': test_data,
        'y_pred_train': y_pred_train,
        'y_pred_test': y_pred_test,
        'train_r2': train_r2,
        'test_r2': test_r2,
        'train_rmse': train_rmse,
        'test_rmse': test_rmse,
        'future_dates': future_dates,
        'future_predictions': y_future,
        'coefficients': model.coef_,
        'intercept': model.intercept_
    }

# Train the model
print("Training linear regression model for evening peak prediction...")
results = train_peak_prediction_model(daily_evening_peaks, train_days=7)

print(f"\n=== Model Performance ===")
print(f"Training R² Score: {results['train_r2']:.4f}")
print(f"Testing R² Score: {results['test_r2']:.4f}")
print(f"Training RMSE: {results['train_rmse']:.2f} kWh")
print(f"Testing RMSE: {results['test_rmse']:.2f} kWh")
print(f"\nModel Coefficients:")
print(f"  - Day number: {results['coefficients'][0]:.4f}")
print(f"  - Day of week: {results['coefficients'][1]:.4f}")
print(f"  - Is weekend: {results['coefficients'][2]:.4f}")
print(f"  - Intercept: {results['intercept']:.2f}")

print(f"\n=== Future Evening Peak Predictions (Next 7 Days) ===")
for date, pred in zip(results['future_dates'], results['future_predictions']):
    print(f"{date.strftime('%Y-%m-%d (%A)')}: {pred:.2f} kWh")
```

```
Training linear regression model for evening peak prediction...

=== Model Performance ===
Training R² Score: 0.9903
Testing R² Score: 0.7259
Training RMSE: 2.73 kWh
Testing RMSE: 15.11 kWh

Model Coefficients:
  - Day number: -1.2170
  - Day of week: -0.0771
  - Is weekend: -61.5525
  - Intercept: 409.74

=== Future Evening Peak Predictions (Next 7 Days) ===
2026-02-19 (Thursday): 392.47 kWh
2026-02-20 (Friday): 391.18 kWh
2026-02-21 (Saturday): 328.33 kWh
2026-02-22 (Sunday): 327.04 kWh
2026-02-23 (Monday): 387.83 kWh
2026-02-24 (Tuesday): 386.54 kWh
2026-02-25 (Wednesday): 385.24 kWh
```

6. Interactive Plotly Dashboard - Live Visualization

Create a comprehensive live dashboard with multiple visualizations showing hourly consumption, smoothed trends, and peak predictions.

```
# Create comprehensive interactive dashboard
fig = make_subplots(
    rows=3, cols=2,
    subplot_titles=(
        'Hourly Consumption - Raw vs Smoothed (MA-6h)',
        'Moving Average Comparison (Different Window Sizes)',
        'Evening Peak Hours Pattern (6 PM - 10 PM)',
        'Daily Evening Peak Trends',
        'Linear Regression: Predicted vs Actual Evening Peaks',
        'Future 7-Day Evening Peak Forecast'
    ),
    specs=[
        [{"secondary_y": False}, {"secondary_y": False}],
        [{"secondary_y": False}, {"secondary_y": False}],
        [{"secondary_y": False}, {"secondary_y": False}]
    ],
    vertical_spacing=0.12,
    horizontal_spacing=0.10
)

# 1. Raw vs 6-hour smoothed consumption
fig.add_trace(
    go.Scatter(
        x=df_smoothed['timestamp'],
        y=df_smoothed['total_consumption_kwh'],
        mode='lines',
        name='Raw Consumption',
        line=dict(color='lightblue', width=1),
        opacity=0.5
    ),
    row=1, col=1
)

fig.add_trace(
    go.Scatter(
        x=df_smoothed['timestamp'],
        y=df_smoothed['total_consumption_kwh_ma6h'],
        mode='lines',
        name='6h Moving Avg',
        line=dict(color='darkblue', width=2)
    ),
    row=1, col=1
)

# 2. Multiple moving averages comparison
fig.add_trace(
    go.Scatter(
        x=df_smoothed['timestamp'],
        y=df_smoothed['total_consumption_kwh_ma3h'],
        mode='lines',
        name='MA-3h',
        line=dict(color='orange', width=1.5)
    ),
    row=1, col=2
)

fig.add_trace(
    go.Scatter(
        x=df_smoothed['timestamp'],
        y=df_smoothed['total_consumption_kwh_ma12h'],
        mode='lines',
        name='MA-12h',
        line=dict(color='green', width=1.5)
    ),
    row=1, col=2
)

fig.add_trace(
    go.Scatter(
        x=df_smoothed['timestamp'],
        y=df_smoothed['total_consumption_kwh_ma24h'],
        mode='lines',
        name='MA-24h',
        line=dict(color='red', width=2)
    ),
    row=1, col=2
)

# 3. Evening peak hours pattern
fig.add_trace(
    go.Scatter(
        x=df_evening['timestamp'],
```

```
        y=df_evening['total_consumption_kwh'],
        mode='markers+lines',
        name='Evening Consumption',
        marker=dict(color='purple', size=4),
        line=dict(color='purple', width=1)
    ),
    row=2, col=1
)

# 4. Daily evening peaks
fig.add_trace(
    go.Scatter(
        x=daily_evening_peaks['date'],
        y=daily_evening_peaks['evening_peak_kwh'],
        mode='markers+lines',
        name='Raw Evening Peak',
        marker=dict(color='orange', size=8),
        line=dict(color='orange', width=2, dash='dot')
    ),
    row=2, col=2
)

fig.add_trace(
    go.Scatter(
        x=daily_evening_peaks['date'],
        y=daily_evening_peaks['evening_peak_smoothed_kwh'],
        mode='markers+lines',
        name='Smoothed Evening Peak',
        marker=dict(color='darkred', size=8),
        line=dict(color='darkred', width=2)
    ),
    row=2, col=2
)

# 5. Predictions vs Actual (Training and Testing)
# Training data
fig.add_trace(
    go.Scatter(
        x=results['train_data']['date'],
        y=results['train_data']['evening_peak_smoothed_kwh'],
        mode='markers',
        name='Training Actual',
        marker=dict(color='blue', size=8)
    ),
    row=3, col=1
)

fig.add_trace(
    go.Scatter(
        x=results['train_data']['date'],
        y=results['y_pred_train'],
        mode='lines',
        name='Training Predicted',
        line=dict(color='lightblue', width=2, dash='dash')
    ),
    row=3, col=1
)

# Testing data
fig.add_trace(
    go.Scatter(
        x=results['test_data']['date'],
        y=results['test_data']['evening_peak_smoothed_kwh'],
        mode='markers',
        name='Testing Actual',
        marker=dict(color='red', size=10)
    ),
    row=3, col=1
)

fig.add_trace(
    go.Scatter(
        x=results['test_data']['date'],
        y=results['y_pred_test'],
        mode='lines',
        name='Testing Predicted',
        line=dict(color='pink', width=2, dash='dash')
    ),
    row=3, col=1
)

# 6. Future predictions (next 7 days)
```

```
fig.add_trace(
    go.Scatter(
        x=results['future_dates'],
        y=results['future_predictions'],
        mode='markers+lines',
        name='Future Forecast',
        marker=dict(color='green', size=12, symbol='star'),
        line=dict(color='green', width=3, dash='dash')
    ),
    row=3, col=2
)

# Add confidence interval for future predictions
std_dev = results['test_rmse']
upper_bound = results['future_predictions'] + 1.96 * std_dev
lower_bound = results['future_predictions'] - 1.96 * std_dev

fig.add_trace(
    go.Scatter(
        x=results['future_dates'],
        y=upper_bound,
        mode='lines',
        name='95% CI Upper',
        line=dict(color='lightgreen', width=1, dash='dot'),
        showlegend=False
    ),
    row=3, col=2
)

fig.add_trace(
    go.Scatter(
        x=results['future_dates'],
        y=lower_bound,
        mode='lines',
        name='95% CI Lower',
        line=dict(color='lightgreen', width=1, dash='dot'),
        fill='tonexty',
        fillcolor='rgba(0, 255, 0, 0.1)',
        showlegend=True
    ),
    row=3, col=2
)

# Update layout
fig.update_xaxes(title_text="Time", row=1, col=1)
fig.update_xaxes(title_text="Time", row=1, col=2)
fig.update_xaxes(title_text="Time", row=2, col=1)
fig.update_xaxes(title_text="Date", row=2, col=2)
fig.update_xaxes(title_text="Date", row=3, col=1)
fig.update_xaxes(title_text="Future Date", row=3, col=2)

fig.update_yaxes(title_text="Consumption (kWh)", row=1, col=1)
fig.update_yaxes(title_text="Consumption (kWh)", row=1, col=2)
fig.update_yaxes(title_text="Consumption (kWh)", row=2, col=1)
fig.update_yaxes(title_text="Peak (kWh)", row=2, col=2)
fig.update_yaxes(title_text="Peak (kWh)", row=3, col=1)
fig.update_yaxes(title_text="Predicted Peak (kWh)", row=3, col=2)

fig.update_layout(
    title_text="<b>Electricity Peak Analysis Dashboard - Dorm Hourly Meter Data</b>",
    title_font_size=20,
    height=1400,
    showlegend=True,
    legend=dict(
        orientation="v",
        yanchor="top",
        y=1,
        xanchor="left",
        x=1.02
    ),
    hovermode='x unified'
)

fig.show()

print("\nLive Plotly Dashboard Generated Successfully!")
```

```
Live Plotly Dashboard Generated Successfully!
```

7. Additional Analysis - Hourly Consumption Heatmap

Create a heatmap showing consumption patterns by hour and day of week.

```
# Create pivot table for heatmap
df_heatmap = df_smoothed.copy()
df_heatmap['day_name'] = df_heatmap['timestamp'].dt.day_name()

# Average consumption by hour and day of week
heatmap_data = df_heatmap.groupby(['day_of_week', 'hour'])['total_consumption_kwh'].mean().reset_index()
heatmap_pivot = heatmap_data.pivot(index='day_of_week', columns='hour', values='total_consumption_kwh')

# Map day numbers to names
day_names = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
heatmap_pivot.index = [day_names[i] for i in heatmap_pivot.index]

# Create heatmap
fig_heatmap = go.Figure(data=go.Heatmap(
    z=heatmap_pivot.values,
    x=heatmap_pivot.columns,
    y=heatmap_pivot.index,
    colorscale='YlOrRd',
    text=heatmap_pivot.values.round(2),
    texttemplate='%{text}',
    textfont={"size": 9},
    colorbar=dict(title="Consumption (kWh)")
))

fig_heatmap.update_layout(
    title='<b>Average Hourly Consumption Pattern by Day of Week</b>',
    xaxis_title='Hour of Day',
    yaxis_title='Day of Week',
    height=500,
    width=1200
)

fig_heatmap.show()

print("\nHeatmap visualization complete!")
```

```
Heatmap visualization complete!
```

8. Summary Statistics and Key Insights

```
print("="*80)
print("ELECTRICITY PEAK ANALYSIS - SUMMARY REPORT")
print("="*80)

# Overall statistics
print("\nOVERALL CONSUMPTION STATISTICS:")
print(f"   Total Records: {len(df_smoothed)}")
print(f"   Date Range: {df_smoothed['timestamp'].min()} to {df_smoothed['timestamp'].max()}")
print(f"   Average Hourly Consumption: {df_smoothed['total_consumption_kwh'].mean():.2f} kWh")
print(f"   Peak Hourly Consumption: {df_smoothed['total_consumption_kwh'].max():.2f} kWh")
print(f"   Minimum Hourly Consumption: {df_smoothed['total_consumption_kwh'].min():.2f} kWh")
print(f"   Standard Deviation: {df_smoothed['total_consumption_kwh'].std():.2f} kWh")

# Evening peak statistics
print("\nEVENING PEAK STATISTICS (6 PM - 10 PM):")
print(f"   Average Evening Peak: {daily_evening_peaks['evening_peak_kwh'].mean():.2f} kWh")
print(f"   Highest Evening Peak: {daily_evening_peaks['evening_peak_kwh'].max():.2f} kWh")
print(f"   Lowest Evening Peak: {daily_evening_peaks['evening_peak_kwh'].min():.2f} kWh")

# Peak hours analysis
peak_hour = df_smoothed.groupby('hour')['total_consumption_kwh'].mean().idxmax()
print(f"\nPEAK CONSUMPTION HOUR: {peak_hour}:00 (Average: {df_smoothed[df_smoothed['hour']==peak_hour]['total_consumption_k

# Weekend vs Weekday
weekday_avg = df_smoothed[df_smoothed['day_of_week'] < 5]['total_consumption_kwh'].mean()
weekend_avg = df_smoothed[df_smoothed['day_of_week'] >= 5]['total_consumption_kwh'].mean()
print(f"\nWEEKDAY vs WEEKEND:")
print(f"   Weekday Average: {weekday_avg:.2f} kWh")
print(f"   Weekend Average: {weekend_avg:.2f} kWh")
print(f"   Difference: {weekday_avg - weekend_avg:.2f} kWh ({((weekday_avg/weekend_avg - 1)*100):.1f}% higher on weekdays)'

# Model performance
```

```
print(f"\nPREDICTION MODEL PERFORMANCE:")
print(f"   Training R² Score: {results['train_r2']:.4f}")
print(f"   Testing R² Score: {results['test_r2']:.4f}")
print(f"   Testing RMSE: {results['test_rmse']:.2f} kWh")
print(f"   Model Accuracy: {results['test_r2']*100:.2f}%")

# Next week forecast
print(f"\nNEXT 7-DAY EVENING PEAK FORECAST:")
for i, (date, pred) in enumerate(zip(results['future_dates'], results['future_predictions']), 1):
    print(f"   Day {i} - {date.strftime('%Y-%m-%d (%A)')}: {pred:.2f} kWh")

avg_future_peak = np.mean(results['future_predictions'])
print(f"\n   Average Predicted Evening Peak (Next Week): {avg_future_peak:.2f} kWh")

print("\n" + "="*80)
print("ANALYSIS COMPLETE - All visualizations and predictions generated successfully!")
print("="*80)
```

```
================================================================================
ELECTRICITY PEAK ANALYSIS - SUMMARY REPORT
================================================================================

OVERALL CONSUMPTION STATISTICS:
   Total Records: 337
   Date Range: 2026-02-05 11:06:10.850909 to 2026-02-19 11:06:10.850909
   Average Hourly Consumption: 289.44 kWh
   Peak Hourly Consumption: 527.53 kWh
   Minimum Hourly Consumption: 50.00 kWh
   Standard Deviation: 153.18 kWh

EVENING PEAK STATISTICS (6 PM - 10 PM):
   Average Evening Peak: 477.94 kWh
   Highest Evening Peak: 527.53 kWh
   Lowest Evening Peak: 409.16 kWh

PEAK CONSUMPTION HOUR: 9:00 (Average: 482.84 kWh)

WEEKDAY vs WEEKEND:
   Weekday Average: 302.85 kWh
   Weekend Average: 255.78 kWh
   Difference: 47.07 kWh (18.4% higher on weekdays)

PREDICTION MODEL PERFORMANCE:
   Training R² Score: 0.9903
   Testing R² Score: 0.7259
   Testing RMSE: 15.11 kWh
   Model Accuracy: 72.59%

NEXT 7-DAY EVENING PEAK FORECAST:
   Day 1 - 2026-02-19 (Thursday): 392.47 kWh
   Day 2 - 2026-02-20 (Friday): 391.18 kWh
   Day 3 - 2026-02-21 (Saturday): 328.33 kWh
   Day 4 - 2026-02-22 (Sunday): 327.04 kWh
   Day 5 - 2026-02-23 (Monday): 387.83 kWh
   Day 6 - 2026-02-24 (Tuesday): 386.54 kWh
   Day 7 - 2026-02-25 (Wednesday): 385.24 kWh

   Average Predicted Evening Peak (Next Week): 371.23 kWh

================================================================================
ANALYSIS COMPLETE - All visualizations and predictions generated successfully!
================================================================================
```